C++ runtime contribution and scope

Tokenomics implements a single-request C++20 speculative-decoding runtime around a pinned llama.cpp. It preserves upstream greedy verification while owning policy decisions, cache reconciliation, retained draft state, lazy catch-up, target-only fallback and request reset.

Implemented contribution

The experimental cost-aware controller selects actions {0, 1, 2, 4, 6, 8} using measured committed tokens per millisecond. Fixed arrays keep its valid select/observe path allocation-free. Calibration, model/binary identity, immutable comparison blocks, independent verification and minimal/detailed telemetry make the result inspectable and reproducible.

Measured result

The completed study has 252 exact validation records: six development prompts, caps 64/128, three repetitions and seven conditions. Cost-aware throughput is 1.2048x cap-8 ngram, 0.9943x the development-frozen fixed-1 baseline and 0.9709x ordinary target-only decoding. It does not establish ordinary-decoding acceleration.

Cost-aware achieves 7.450 tokens/s. Median warm completion is 20.43s versus 19.98s for ordinary decoding. Peak lifetime RSS is 2.18 versus 2.17 GiB, including load/warm-up.

Baseline	Decode t/s	Cost-aware / baseline (95% CI)
Ordinary target	7.673	0.9709 [0.9607, 0.9810]
Frozen fixed-1	7.492	0.9943 [0.9853, 1.0023]
Ngram cap-8	6.183	1.2048 [1.1271, 1.2868]
Acceptance heuristic	7.252	1.0272 [1.0157, 1.0403]

Operating point and separation

Qwen2.5-3B-Instruct Q4_K_M with ngram-simple drafting; context 2048, batch 512, greedy temperature-only sampling, F16 KV and two inference threads. The allocated Linux Codespace has two logical CPUs, reports an AMD EPYC 7763 topology and about 7.8 GiB RAM, with no swap or GPU. The three calibration prompts are separate from validation. Rates are geometric means of cell median throughput. Original synthetic CC0 prompts are a controlled workload, not a quality benchmark.

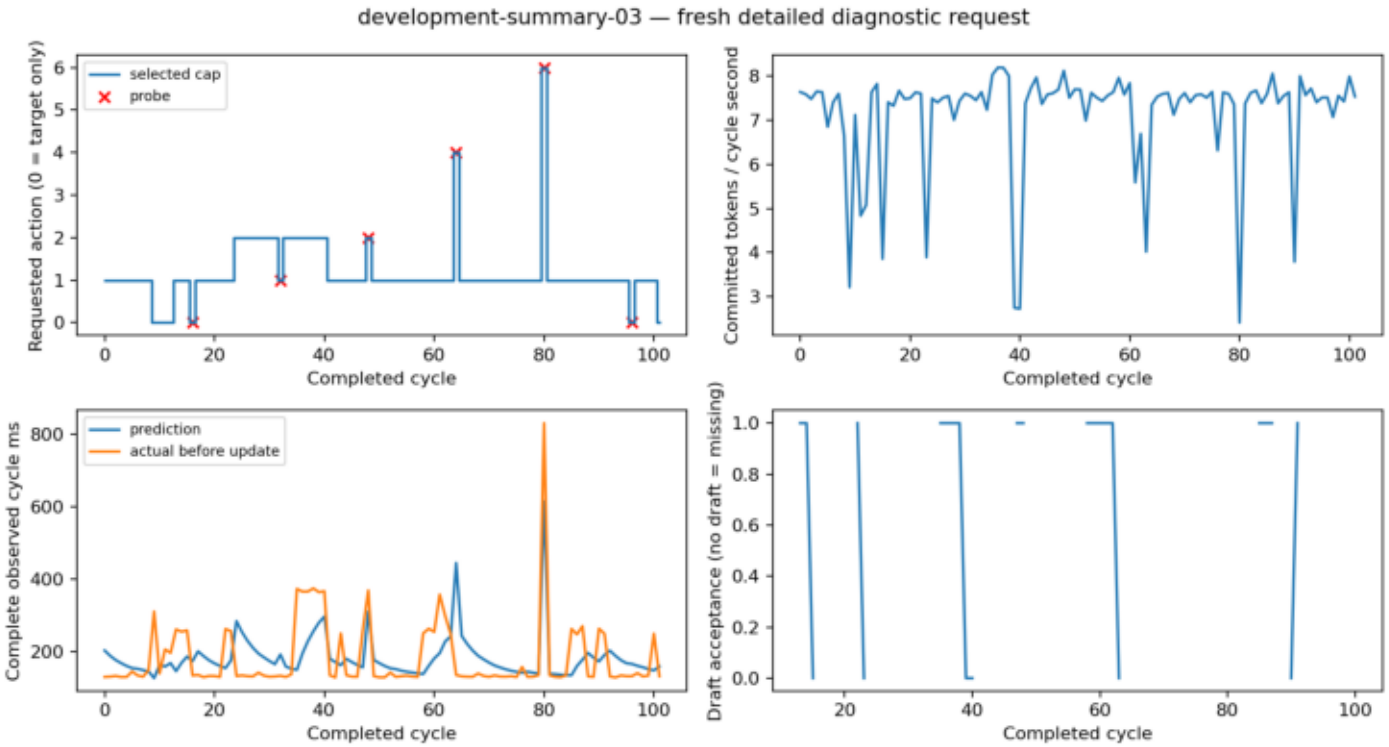
The controller was completed experimentally despite negative feasibility evidence. This checkpoint is not full Pilot A/B, the 90-prompt held-out evaluation, a Mac result or a novelty claim. A separate CPU-profile experiment cannot alter this frozen guarded-profile result.

Controller and cache-state design

At each cycle boundary, cached prefix P plus pending token L is the exact committed history. The target evaluates L and up to k proposed tokens. Upstream returns a matching prefix and a correction/bonus; Tokenomics commits only through EOG or budget, trims invalid suffixes and checks cache positions. Retaining valid draft KV avoids replaying rejected suffixes. Full acceptance can still leave draft state stale; catch-up is charged before re-entry.

Score(a) = estimated committed tokens / (estimated base milliseconds + full transition cost). Chosen-action-only EWMA updates use alpha 0.2. Actual positive-token synchronization is removed from base cost to avoid double counting and is learned per synchronized token. A four-normal-cycle hold and 5% switch margin limit oscillation; alternate probes rotate every 16 completed cycles. Request and warm-up state reset to frozen development priors. Calibration restores original target KV checkpoints and identical prepared draft state; that diagnostic setup is not deployable runtime work.

Across 36 measured requests, ordinary fallback supplies 47.3% of 3456 output tokens. There are 175 probes and 99 normal incumbent switches. A particular alternate action may wait 80 cycles under an unchanged incumbent; this is a lightweight experimental estimator, not an optimal scheduler.

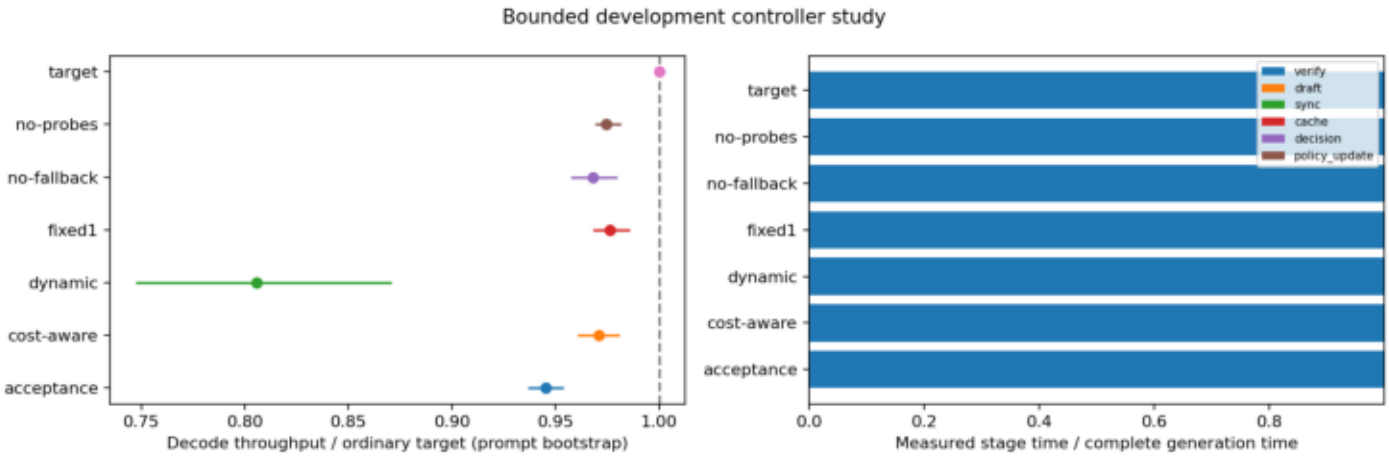


Declared worst-regression prompt, measured again with detailed logging. This is a fresh diagnostic history, not the original minimal timing trace. Probes, fallback, prediction error and acceptance remain visible.

Correctness, overhead and mechanism

Both guarded and packed Cloud Release and ASan/UBSan/leak checks pass six suites, including 906 differential fixture cases. Trained controller gates pass 28 records for each drafting mechanism and exercise all six actions; model drafting observes 40 synchronized tokens. Those focused gates use explicitly synthetic priors to force behavior, and their timings are excluded from performance claims.

All 252 validation and 16 separate logging records pass exact token IDs, stops and cache accounting. The first attempt paused after 25/36 blocks; unchanged resume preserves every earlier raw block, fit measurement and prior byte-for-byte. All 1,587 payload files plus the final manifest are retained with ZIP/inner-archive hashes and actual Shutdown evidence.



Measured overhead and work

Decision plus update averages 1.67 microseconds per cycle, 0.00114% of generation. Mean initialization is 0.83 microseconds; parsing is 8.63 microseconds per request and remains inside warm latency. Mean absolute fractional cycle-time prediction error is 22.7%.

For the same 3456 output tokens, target-only evaluates 3456 target rows, cost-aware 3604 and cap-8 ngram 4572. Cost-aware avoids 21.2% of cap-8 target work but still evaluates 4.3% more rows than ordinary decoding. Verification occupies 99.992% of measured generation. Removing all other time with verification held fixed could improve throughput by only about 0.0083%.

No-fallback/cost-aware is 0.9970 [0.9933, 1.0005]; no-probes/cost-aware is 1.0036 [0.9953, 1.0118]. Detailed/minimal logging is 1.0015 [0.9868, 1.0165] on the separate two-prompt grid. None shows a practical aggregate gain. Probe execution uses 6.89% of generation, but that is full probe time including useful committed output, not incremental overhead.

Statistics collapse repetitions before geometric-mean pairing and resample distinct prompts with both caps together. Repeats are not independent workload samples. All twelve fixed-1 comparison cells are within the declared 5% practical margin; no cost-aware/ordinary cell is more than 10% slower.

Reproduce, interpret and extend

Native CPU optimization

The verification bottleneck motivates a separate row-consistent packed CPU profile. Local correctness and sanitizer checks pass, but this report includes no completed trained packed-profile performance result. A prerequisite gate, paired component comparison and new controller calibration remain separate from the guarded measurements above.

Build and inspect without model inference

```
git clone --recurse-submodules https://github.com/VictorWong123/tokenomics.git
cd tokenomics && bash scripts/bootstrap.sh cpu
.venv/bin/python scripts/demo_controller.py --max-cycles 12
.venv/bin/python -m analysis.engineering_report --out results/local/report
# Include a separately verified CPU study with --profile-receipt PATH, if available.
```

The recorded demo verifies the retained trained archive and replays real decisions. For new model runs, use the documented owned Codespace workflow, exact weights/lock files and a freshly calibrated operating point. Commands, detailed CLI contracts, analysis and frozen grids are in README.md, docs/controller-design.md, docs/controller-results.md and docs/evaluation.md.

Provenance and ownership

Controller inference source: e98c0fa84a2c2ff52df198f19eae6ce72d51c83c. Run 42492089188, attempt 2. llama.cpp pin: fd27b2034905f8dadd8bd6d2cc44595f4929d391. The canonical receipt records full executable, target, archive and verifier hashes. Raw measurements remain immutable; tables and this PDF are reproducible derived artifacts.

llama.cpp supplies execution, tokenization, draft generation and common_sampler_sample_and_accept_n. Tokenomics owns policy/state integration, cache reconciliation, budgets, calibration, tests and measurement tooling. The packed profile adapts dispatch/layout around existing native CPU kernels; it does not claim a new GPU kernel. Upstream software and model licenses remain their own; Qwen2.5-3B has Qwen Research terms. No weights or credentials are committed.

Limits and next evidence

Full 90-prompt held-out model evaluation and Mac/Metal execution remain unexecuted. The bounded development results do not establish universal acceleration, model quality, energy or dollar savings, stochastic correctness or research novelty. Prior cost estimates can become stale; transition charging is conservative over one cycle. A different executable/backend needs new calibration and its own ordinary baseline.

Primary references and implementation differences are documented in docs/related-work.md and docs/upstream.md, including Leviathan et al. (2023), Chen et al. (2023), dynamic-lookahead work and the stock executable's different stopping/counting endpoint. The next optimization follows measured costs rather than interpreting high acceptance alone as speedup.